National University of Computer & Emerging Sciences
Karachi Campus



WORDLE GAME

Project Proposal

Data Structures

Section: K

Group Members:
25k-3011 Ali
25k-2513 Ali Hussain
25k-0906 M. Abbas

Project Proposal

- **Introduction**

  Wordle is a word-guessing game in which the player has six attempts to find a hidden five-letter word. After every guess, each letter is marked as correct and in the right place, correct but in the wrong place, or not in the word. Our project is a Wordle game developed in C++ using Object Oriented Programming and a graphical user interface (GUI). Its purpose is to show how data structures, mainly the Binary Search Tree (BST), arrays and stacks, can be applied to a real game to store, search and manage data efficiently.

- **Existing System**

  We studied an open-source C++ implementation, Wordle-in-Cpp (github.com/Loudhabs/Wordle-in-Cpp), which we will use as our reference system:

  - It is a console (text-based) game. The player types a guess and the result is printed as G (correct place), Y (wrong place) or - (not in the word).
  - The word list (ListOfWords.txt) is read into a sorted vector, a five-letter secret word is picked randomly, and each guess is validated against the list using binary search.
  - The player has six attempts per game.
  - It has three classes: Wordle (game logic, attempt count, input check, letter comparison, win detection), Tree (builds a balanced BST from a string and searches it) and Node (the tree node).
  - The secret word is stored in a balanced BST so that checking whether it contains a given letter is efficient.

- **Problem Statement**

  The existing system is a good base but has limitations:

  - It has no graphical interface, so the player cannot see colored tiles or a keyboard like the original Wordle.
  - The BST holds only the five letters of the secret word, so the data structure is used on a very small amount of data.
  - The player gets no help while playing: no record of letters already tried, no hints and no choice of difficulty.
  - There is no score, streak or statistics tracking, so nothing is kept between games.

- **Proposed Solution**

  We will keep the reference classes as our foundation and add the following:

  - **GUI:** a window built with the Raylib library showing a 6x5 grid of tiles that turn green, yellow or gray after each guess, with keyboard typing and Enter / Backspace support.
  - **On-screen keyboard and letter tracker:** an array of 26 elements storing the status of each letter (unused, absent, misplaced, correct), drawn as a keyboard under the grid.
  - **Guess history:** a stack that stores previous guesses with their feedback.
  - **Hint system:** uses the BST to reveal one letter of the secret word at the cost of a score penalty.
  - **Difficulty levels:** easy (more attempts), normal (six attempts) and hard (revealed hints must be reused).

- **Score and leaderboard:** score based on attempts used, saved to a file and sorted by score.
- **Repeated letters:** correct green / yellow marking when a guess contains duplicate letters.

- **Salient Features**
  - Graphical Wordle board with colored tiles and keyboard input
  - Random secret word selection and six-attempt gameplay
  - Dictionary validation using binary search on a sorted list
  - Balanced BST for letter lookup in the secret word
  - On-screen keyboard (array) and guess history (stack)
  - Hint system with score penalty and three difficulty levels
  - Score tracking and sorted leaderboard saved in a file
  - Object Oriented design with separate Wordle, Tree and Node classes
- **Tools & Technologies**
  - **Programming language:** C++ (Object Oriented Programming)
  - **GUI library:** Raylib
  - **Data structures:** BST, arrays, stack, sorted list with binary search
  - **Storage:** text files for the word list and scores
  - **IDE / compiler:** Visual Studio / VS Code / Dev-C++ (g++)
  - **Version control:** Git and GitHub
  - **Operating System:** Windows